Part 11: Model Selection

Previously, we had discussed the topic of **model selection** within the context of regression, i.e., for deciding which predictors to include in a regression model.

Here we will broaden this to cover a more general class of models, e.g., how do we select from among a list of candidate models? E.g., Normal versus t versus ...

If we only seek to maximize the likelihood over candidate models, we will be led to **overfitting** to the sample, since a model with more complexity (more parameters, more flexibility), will necessarily fit better to the data.

Fortunately, we can continue to use AIC and related tools for this purpose.

Example: Alternatives to the Lognormal Pricing Model

Here, let S_t denote the closing price of some equity on day t.

If we assume that $S_t = S_{t-1}X_t$, where X_t has the lognormal(μ, σ^2) distribution, i.e., $\log(X_t)$ has the Normal(μ, σ^2) distribution. Further assume that $X_1, X_2, \ldots$ are independent.

This is the **lognormal pricing model** that underlies a lot of financial theory, including Black-Scholes.

One can motivate this model by assuming that $X_t = Y_1 Y_2 \cdots Y_m$, where the Y_i are i.i.d. The CLT states that $\log(X_t)$ will be approximately normal.

This model is known to not fit well to real data. In particular, log returns have heavier tails than the normal distribution.

Consider the returns for Kellogg's (K) from 2010 to 2018:

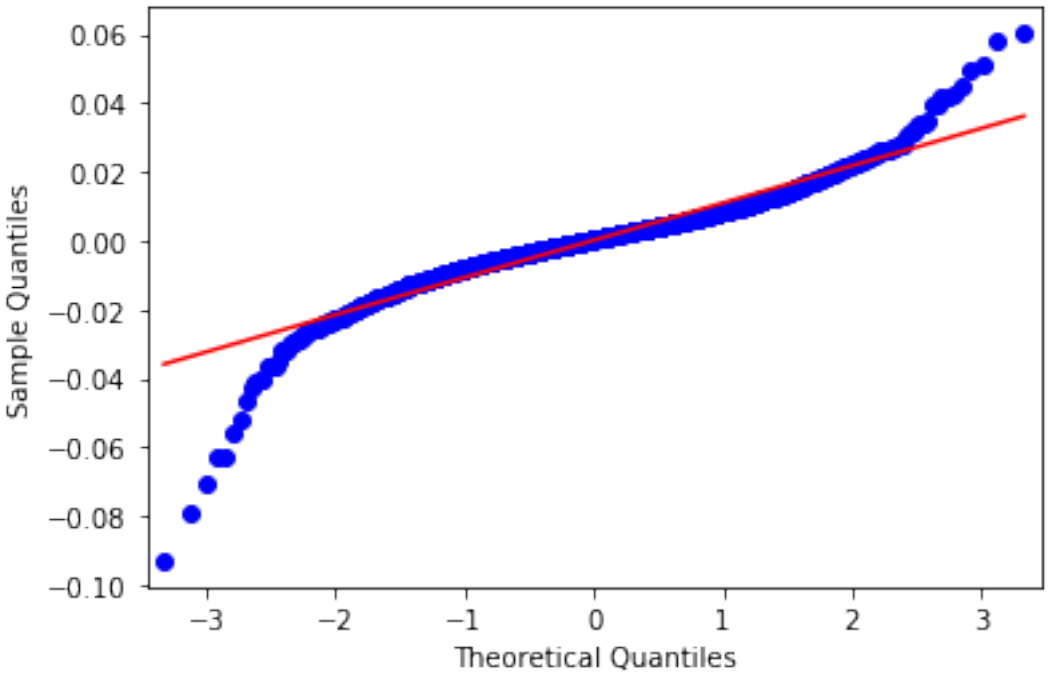
```
In [1]: import yfinance as yf
        import numpy as np
        Kdat = yf.download("K", start="2010-01-01",
           end="2018-12-31")
        ldrK = np.log(Kdat['Adj Close']).diff()[1:]

[**********************100%************************]  1 of 1 down
```

The following normal probability plot shows the deficiencies in the normal assumption.

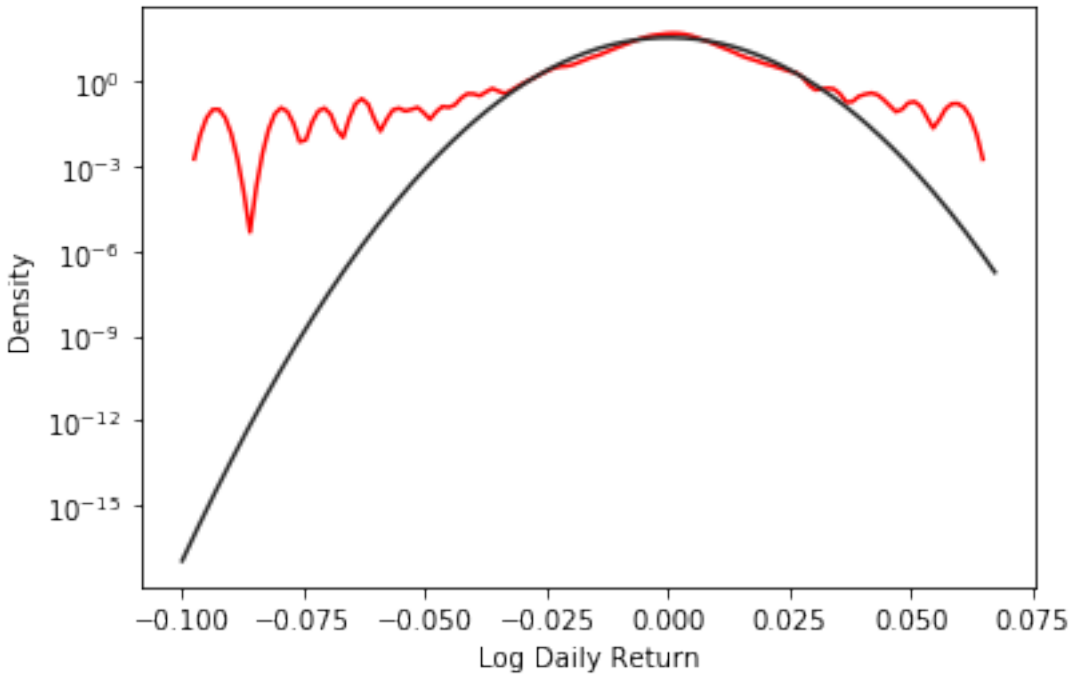
```
In [5]: from statsmodels.graphics.gofplots import qqplot
        import scipy.stats.distributions as spdists

        qqplot(ldrK,line="s")
        plt.show()
```



Another way to highlight the problem with the normal fit is to compare the **kernel density estimate (KDE)** with the best-fitting normal distribution. The KDE is a **nonparametric** approach to estimating a density, meaning that it does not make any assumptions regarding the form of the density.

Note how the log scale is used to emphasize the problems in the tails.



The deficiencies in the normal distribution have led people to consider alternatives. Here we will look at a few and then compare their fit to our return data.

The Generalized Error Distribution (GED)

(This is sometimes referred to as the **generalized normal distribution**.)

Random variable X has the GED with mean μ, scale α, and shape β if

$$f_X(x) = \frac{\beta}{2\alpha\Gamma(1/\beta)} \exp\left[\left(-\frac{|x-\mu|}{\alpha}\right)^{\beta}\right], \quad -\infty < x < \infty, \;\; \beta > 0, \;\; \alpha > 0$$

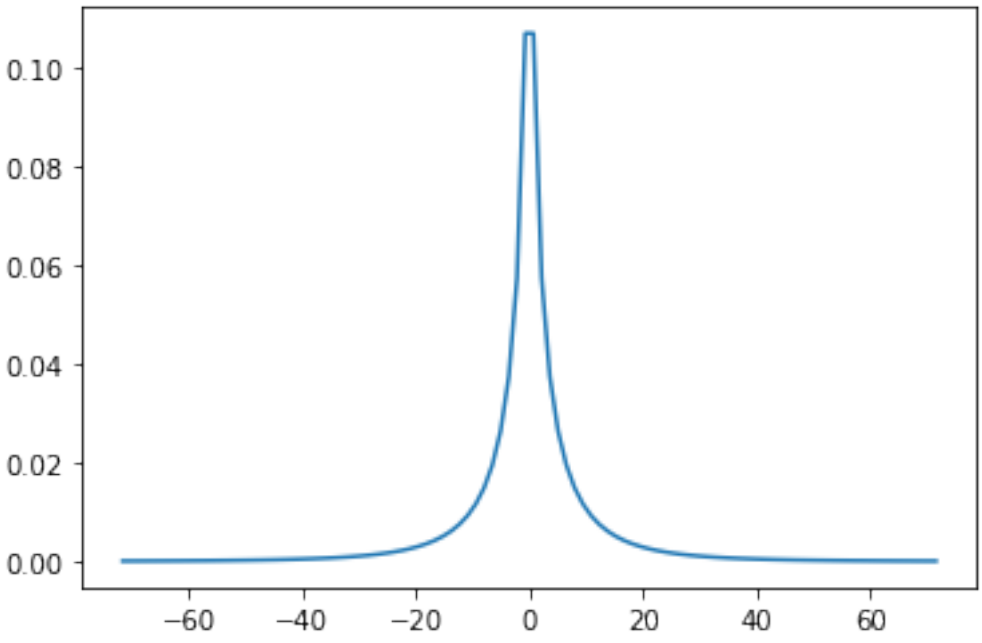
Comments:

1. When $\beta = 2$, this is the normal$(\mu, \alpha^2/2)$ distribution. When $\nu = 1$, this is the **double exponential** or **Laplace** distribution.
2. The distributions are symmetric around μ.
3. Package `scipy.stats` includes `gennorm`.

Exercise: Use the code below to explore the effect of varying β on the shape of the pdf of this distribution.

```
In [7]: from scipy.stats import gennorm

        beta = 0.5
        x = np.linspace(gennorm.ppf(0.001, beta),
            gennorm.ppf(0.999, beta), 100)
        plt.plot(x, gennorm.pdf(x, beta))
        plt.show()
```


0.10
0.08
0.06
0.04
0.02
0.00
−60
−40
−20
0
20
40
60

The Nonstandard t-distribution

Assume T has the (standard) t-distribution with ν degrees of freedom. Then define

$$X = \mu + \sigma T \tag{1}$$

where $\sigma > 0$.

This distribution with have the "shape" of the t-distribution, but will have mean μ and scale σ. We will call this the **nonstandard t-distribution**.

In Python, this distribution is implemented via `scipy.stats.t`.

Exercise: What is the variance of a random variable with this distribution?

Theodossiou's Skewed, Generalized t Distribution

From Theodossiou, "Financial Data and the Skewed Generalized T Distribution," *Management Science*, 1998.

This distribution always has its mode (peak) at zero, but can be **skewed (asymmetric)**.

There are four parameters:

1. σ^2, the variance. $\sigma^2 > 0$
2. k, which controls the height at the peak. $k > 0$
3. n, which controls the probability in the tails. $n > 2$
4. λ, which controls the amount of skew. $-1 < \lambda < 1$

When $\lambda = 0$ and $k = 2$, this matches the nonstandard t distribution with n degrees of freedom, variance σ^2, and $\mu = 0$.

Equation (7) in the paper gives the mean of the distribution. As is often the case, more complexity in the distribution leads to more complexity in the expressions for quantities such as the mean.

We could easily generalize this distribution a bit further, and say that X has the **shifted, skewed generalized T distribution** with shift parameter μ if $X - \mu$ has the non-shifted distribution. Note that $E(X) = \mu$ if and only if $\lambda = 0$.

This notebook includes functions for working with this distribution. (These are not shown in the pdf version.)

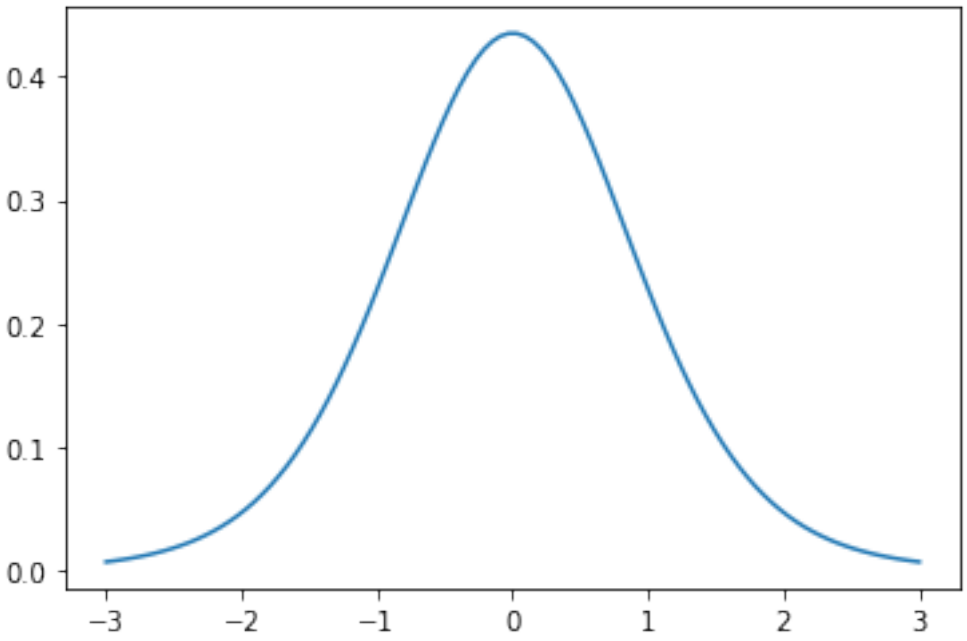
There are a pair of functions that evaluate the density of this distribution. The first is called `dSkewT()` and uses the $(k, n, \lambda, \sigma^2, \mu)$ parameterization. The second is `dSkewTb()`, and uses the (V, k, n, Λ, μ) parameterization. Both of these functions include an argument `log` which enables the user to obtain the log density.

Exercise: See Equations (26) and (27) in the paper (page 1655). What is the relationship between the two parameterizations, and what is the motivation for considering the reparameterization?

Exercise: Use the code below to explore the shape of the density as a function of the five parameters.

```
In [10]: k = 2
         n = 10
         lam = 0
         sigma2 = 1
         mu = 0

         x = np.linspace(mu-3*np.sqrt(sigma2),
             mu+3*np.sqrt(sigma2), 100)
         plt.plot(x, dSkewT(x,k,n,lam,sigma2,mu))
         plt.show()
```



Another function, `FitSkewT()`, returns the MLE of the parameters of the distribution given a vector of observations. Setting `allowshift=True` will fit the μ parameter along with the other four. The function uses `dSkewTb()` to evaluate the log likelihood, but returns the MLE in the $(k, n, \lambda, \sigma^2, \mu)$ parameterization.

Exercise: Try the code below to test the function.

```
In [12]: from scipy.stats import t

         x=t.rvs(df=15,size=1000)

         FitSkewT(x, allowshift=True)

Out[12]: array([ 2.50942308,  7.8861331 ,  0.07165474,  1.13416
```

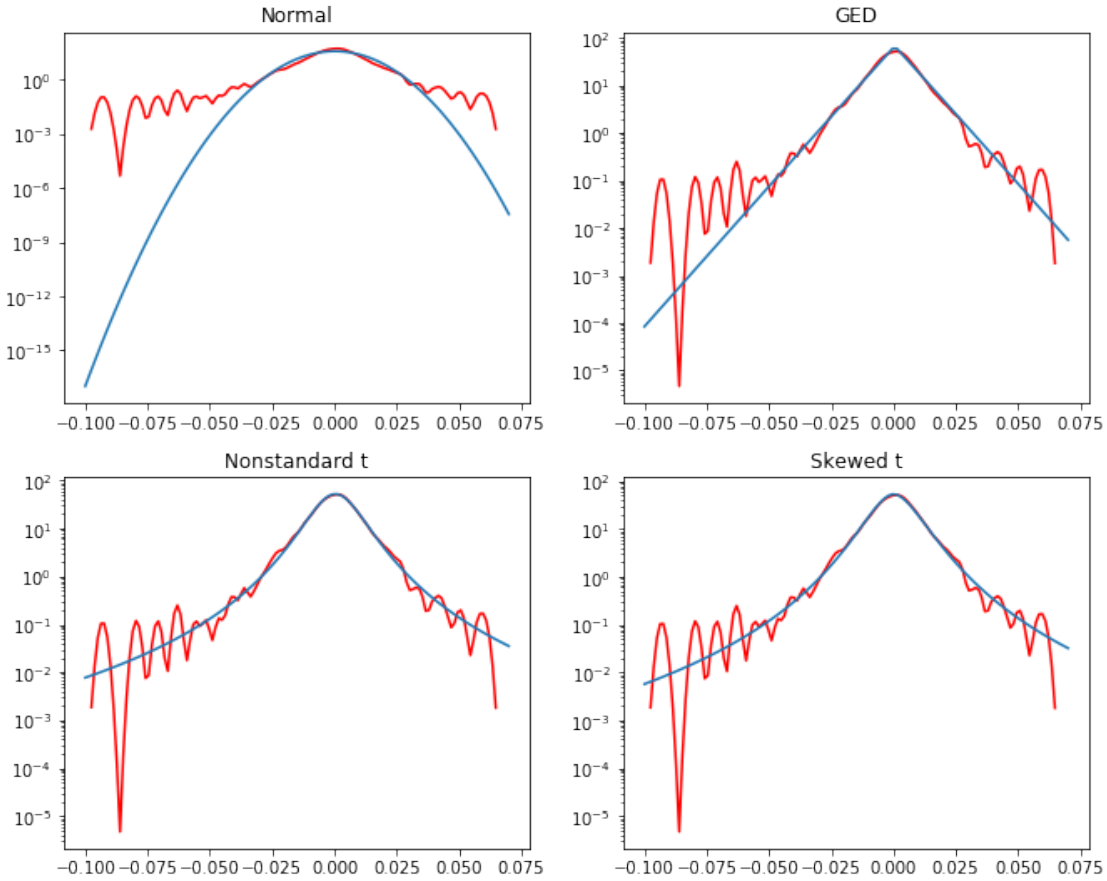
Fitting the Models

We will use maximum likelihood to fit all of the proposed models to the Kellogg's data. The following Python functions make this easy:

```
In [13]: normalfit = norm.fit(ldrK)
         GEDfit = gennorm.fit(ldrK)
         tfit = t.fit(ldrK)
         skewTfit = FitSkewT(ldrK)
         skewTshiftfit = FitSkewT(ldrK, allowshift=True)

In [14]: GEDfit

Out[14]: (1.0068367505002245, 0.00045938387621679205, 0.0075271
```


Normal
GED
Nonstandard t
Skewed t
−0.100 −0.075 −0.050 −0.025 0.000 0.025 0.050 0.075

Calculate the log likelihood for each of the five models:

```
In [16]: print(sum(norm.logpdf(ldrK,normalfit[0],
             normalfit[1])))
         print(sum(gennorm.logpdf(ldrK,GEDfit[0],
             GEDfit[1],GEDfit[2])))
         print(sum(t.logpdf(ldrK,tfit[0],tfit[1],
             tfit[2])))
         print(sum(dSkewT(ldrK,skewTfit[0],skewTfit[1],
             skewTfit[2],skewTfit[3],log=True)))
         print(sum(dSkewT(ldrK,skewTshiftfit[0],
             skewTshiftfit[1],skewTshiftfit[2],
             skewTshiftfit[3],skewTshiftfit[4],log=True)))
```

```
7025.0977903465455
7251.410021251374
7269.972120794004
7268.690331122752
7271.711172130996
```

Exercise: Should these five models be ranked using their likelihoods?

Overfitting

Model 1 is **nested** in Model 2 if Model 1 is a special case of Model 2. In this case, the likelihood maximized under Model 2 will necessarily be greater than or equal to the likelihood maximized under Model 1.

In our example, Normal is nested in the GED and the nonstandard t distribution is nested in the skewed, generalized t distribution with drift.

But: Increased likelihood may not be a sign of a better fitting model. It is possible that we are simply **overfitting** to the available sample.

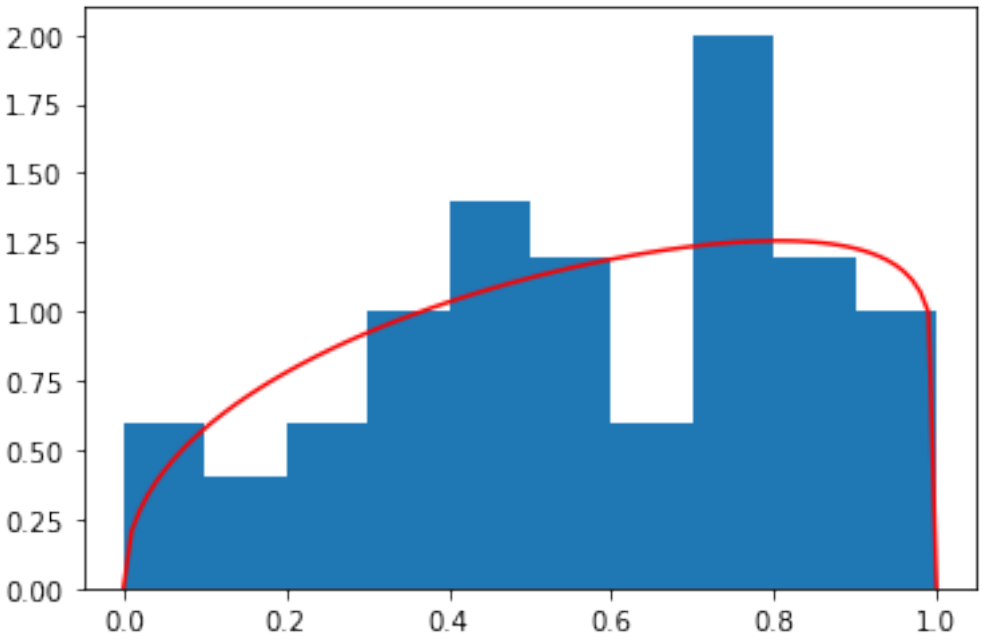
Our objective is to fit well to the **population**, not just to the observed sample. The sample may contain **artifacts** which appear to be real features, but would disappear with a larger sample size.

Example: Suppose that a sample of size $n = 50$ is drawn from the Uniform$(0, 1)$ distribution. If we inspect the histogram of this sample, it is not going to be "flat" due to natural variability.

We may be tempted to fit a Beta(α, β) distribution to this sample, in an effort to fit to any "features" we observe. This would be a mistake, and would be overfitting.

See the figure below. Maximum likelihod is used to fit the Beta distribution to the uniform data.

Exercise: Label the "artifacts" in this histogram.



Models with more parameters have greater flexibility to fit to features, but also artifacts, in observed data. Hence, the risk of overfitting increases greatly as we consider models with a larger number of parameters (and more complexity).

Akaike (1973) proposed a simple way to balance likelihood and model complexity, and this has proven to be a very valuable tool for **model selection**. Define

$$\text{AIC} = -2 \log L(\theta) + 2p$$

where p is the number of parameters in the model. We seek the model that has the **smallest** value of AIC.

Think of $2p$ as being a "penalty" on the likelihood for the complexity of the model.

Exercise: Which of our five candidate models is selected by AIC in the Kellogg's example?

```
In [18]: print(-2*sum(norm.logpdf(ldrK,normalfit[0],
             normalfit[1]))+4)
         print(-2*sum(gennorm.logpdf(ldrK,GEDfit[0],
             GEDfit[1],GEDfit[2]))+6)
         print(-2*sum(t.logpdf(ldrK,tfit[0],tfit[1],
             tfit[2]))+6)
         print(-2*sum(dSkewT(ldrK,skewTfit[0],skewTfit[1],
             skewTfit[2],skewTfit[3],log=True))+8)
         print(-2*sum(dSkewT(ldrK,skewTshiftfit[0],
             skewTshiftfit[1],skewTshiftfit[2],
             skewTshiftfit[3],skewTshiftfit[4],log=True))+10)
```

```
-14046.195580693091
-14496.820042502748
-14533.944241588008
-14529.380662245503
-14533.422344261991
```